Chapter 1: Simple Expressions

We can begin building a programming language with one expression:

```
3+4*5
```

We expect `23`. Getting there requires decisions that are easy to overlook when we already know arithmetic. Which characters belong together? Which operation applies to which values? Where does multiplication's higher precedence come from?

The Vertex interpreter in `chapter-01-simple-expressions/code` makes those decisions explicit. It recognizes numbers and the four basic arithmetic operators. Parentheses let us change the grouping. That is enough language to study how source text acquires structure and how that structure determines a result.

This chapter follows `runner.py`, `tokenizer.py`, `parser.py`, and `evaluator.py`. Code excerpts come from those files, with occasional formatting adjustments. Additional examples are identified as such.

Running an Expression

From this chapter's code directory, run:

```
./vertex '3+4*5'
```

The result is:

```
23
```

Quote the expression so the shell passes it as one argument and does not interpret characters such as `*`. The runner also accepts a filename ending in `.v` or `.t`:

```
./vertex example.v
```

The supplied file contains `-1.5+3*2`, which uses both floating-point literals and unary minus. The runner reads the whole file as one expression, rather than executing a sequence of statements. Whitespace can separate tokens, but a newline does not begin a new statement in this version.

`./vertex` launches the Python runner and passes along its arguments. The runner prints the answer using Python. Vertex does not yet have its own output statement. There are no variables or assignments here either. Those require more language than we have implemented so far.

Three Stages of Interpretation

The runner connects the stages:

```
tokens = tokenize(expression)
ast = parse(tokens)
result = evaluate(ast)
print(result)
```

The tokenizer turns characters into tokens. The parser organizes those tokens into an **abstract syntax tree (AST)**. The evaluator computes the value represented by that tree.

Stage	Input	Output
Tokenizer	Source text	List of token dictionaries
Parser	Token list	Tree of node dictionaries
Evaluator	Expression tree	Numeric value

These stages answer different questions. The tokenizer can recognize every character in `3+*4`, but that does not make it a valid expression. The parser rejects the arrangement. Conversely, `3/0` has a valid arrangement but fails when evaluated.

The separation also gives us useful places to inspect a problem. A wrong numeric token calls for a look at tokenization. A wrong grouping calls for a look at parsing. Once the tree is correct, we can examine how evaluation handles its nodes.

Recognizing Tokens

The tokenizer uses regular expressions, which describe patterns in text. (1) Its patterns begin:

```
patterns = [
    (r"\s+", "whitespace"),
    (r"//[^\r\n]*", "comment"),
    (r"\d*\.\d+|\d+\.\d*|\d+", "number"),
    (r"\+", "+"),
    (r"\-", "-"),
    (r"\/", "/"),
    (r"\*", "*"),
    (r"\(", "("),
    (r"\)", ")"),
    (r".", "error"),
]
```

Each pair associates a pattern with a tag. The `r` prefix makes a Python raw string, which is convenient when writing patterns containing backslashes. The patterns are compiled with `re.compile` before scanning begins.

At the current position, the tokenizer tries each pattern in order:

```
for pattern, tag in patterns:
    match = pattern.match(characters, position)
    if match:
        current_tag = tag
        break
```

`match` starts at the supplied position. It does not search ahead and skip characters it cannot recognize. The first matching pattern wins. The final error pattern catches an otherwise unrecognized character, such as `@`.

The number pattern recognizes both integer and floating-point literals:

```
r"\d*\.\d+|\d+\.\d*|\d+"
```

Here, `\d` matches a digit, and `+` means one or more repetitions. The first two alternatives recognize decimal forms such as `.5`, `5.`, and `12.5`. They appear before the integer alternative so that the tokenizer does not stop after only the initial digits in a source value such as `12.5`.

The minus sign is a separate token. The tokenizer does not treat `-5` as one signed number. Whether a minus token can appear before an expression is a question for the grammar.

Token Values and Positions

For input other than whitespace or comments, the tokenizer constructs a dictionary:

```
token = {"tag": current_tag, "line": line, "column": column}
if current_tag == "number":
    if "." in value:
        token["value"] = float(value)
    else:
        token["value"] = int(value)
tokens.append(token)
```

A numeric token carries a Python number rather than just the original text. For example, `"3"` becomes the integer `3`, while `"12.5"` becomes the floating-point value `12.5`. Operator tokens carry their tag and source location but do not need a numeric value.

For the source `3+4*5`, tokenization produces:

```
[
    {"tag": "number", "line": 1, "column": 1, "value": 3},
    {"tag": "+", "line": 1, "column": 2},
    {"tag": "number", "line": 1, "column": 3, "value": 4},
    {"tag": "*", "line": 1, "column": 4},
    {"tag": "number", "line": 1, "column": 5, "value": 5},
    {"tag": None, "line": 1, "column": 6}
]
```

Whitespace and comments are consumed without becoming tokens. The tokenizer still updates the line and column as it advances, so later tokens keep their locations in the original input. Lines and columns start at one.

The final token has a tag of `None`. It marks the end of input. That gives the parser something explicit to inspect when no more ordinary tokens remain. It is a token dictionary whose tag is `None`, rather than a bare `None` in the list.

Discarding Comments

A Vertex line comment starts with `//` and continues to the end of the line:

```
8 / 2 // Divide eight by two.
```

The comment pattern is `r"//[^\r\n]*"`. The two slashes match literally. The bracket expression `[^\r\n]` matches any character except a carriage return or newline, and `*` allows zero or more of those characters. This also handles a bare `//` and a comment at the end of a file without a final newline.

Pattern order matters. The comment pattern comes before the division pattern, so two adjacent slashes start a comment rather than becoming two division tokens. A single slash still means division. The tokenizer consumes the whole comment in one match; punctuation inside it is not examined as program text.

The matched text receives the tag `"comment"`, but this condition keeps it out of the token list:

```
if current_tag not in ("whitespace", "comment"):
```

The line ending is left for the whitespace pattern. Position tracking still runs for the discarded text. The parser and evaluator never see the comment, so neither needs a comment rule. The expression above produces the same tokens and result as `8 / 2`.

A comment can also occupy its own line or follow part of an unfinished expression:

```
// One expression, continued on the next line.
8 + // Add the product below.
2 * 3
```

After comments and whitespace are discarded, this is `8 + 2 * 3`, which evaluates to `14`. Ending a comment does not end the expression.

Describing the Grammar

The parser file gives the grammar in Extended Backus-Naur Form (EBNF). (2)

```
expression ::= term { ("+" | "-") term }
term ::= unary { ("*" | "/") unary }
unary ::= "-" unary | factor
factor ::= <number> | "(" expression ")"
```

An expression starts with a term and may continue with additions or subtractions. A term starts with a unary expression and may continue with multiplications or divisions. A unary expression is either a leading minus sign followed by another unary expression, or a factor. A factor is either a number or a parenthesized expression.

Braces mean that the enclosed part can repeat zero or more times. The vertical bar separates alternatives. Quoted symbols are actual tokens in the language; the unquoted grammar names describe larger pieces.

A single number is therefore a factor, a unary expression, a term, and an expression. We do not need an operator for a valid expression. The separate unary rule also makes negation work in places where a full operand is expected, such as `-5`, `3*-2`, and `-(3+4)`.

The grammar also explains precedence. An expression must obtain complete terms before combining them with `+` or `-`. A multiplication inside a term is grouped before the surrounding addition. This decision belongs to the parser, not to a later search for the most important operator.

Building an Expression Tree

Our AST uses a number node for a literal:

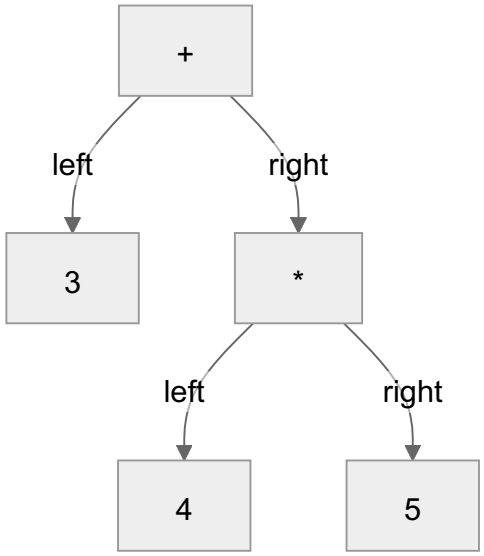
```
{"tag": "number", "value": 3}
```

A binary operator node has an operator tag and two children:

```
{
    "tag": "+",
    "left": {"tag": "number", "value": 3},
    "right": {"tag": "number", "value": 4}
}
```

The children are themselves nodes. Nesting them represents larger expressions without having to put the source text back together.

The Tree for 3+4∗5



The multiplication belongs to the right child of the addition. Its result supplies one operand of +.

The tree is abstract because it keeps the structure needed for computation without retaining every detail of the source. (3) There are no whitespace nodes. Parentheses influence the tree but do not survive as separate nodes in this implementation. The numeric AST nodes also omit the source positions that were present in the tokens.

Parsing a Factor

A recursive-descent parser uses cooperating functions to recognize grammar rules. (4)

A recursive-descent parser uses functions corresponding to grammar rules. Here, `parse_expression` calls `parse_term`, which calls `parse_unary`, which calls `parse_factor`. Parentheses allow `parse_factor` to call `parse_expression` again.

Each parsing function returns two things: the node it built and the tokens it has not consumed. The number case in `parse_factor` is:

```
token = tokens[0]
if token["tag"] == "number":
    node = {"tag": "number", "value": token["value"]}
    return node, tokens[1:]
```

Reading 3+4 as a factor consumes the 3 and leaves the +, the 4, and the end token. Returning the unused tokens lets the calling function continue at the correct place. These slices create new lists; the parser is not removing elements from the original list in place.

The parenthesis case does more work:

```
if token["tag"] == "(":
    node, tokens = parse_expression(tokens[1:])
    if tokens[0]["tag"] != ")":
        raise SyntaxError(f"Expected ')', got {tokens[0]}")
    return node, tokens[1:]
```

It skips the opening parenthesis and parses a complete expression inside it. Then it requires and consumes the closing parenthesis. The returned node is the inner expression's node. For `(3+4)*5`, that addition becomes the left operand of multiplication.

If the next token is neither a number nor an opening parenthesis, the factor function raises a syntax error. This catches an operator where an operand was required, including the second operator in `3+*4`.

Parsing Unary Minus

Unary minus is handled between terms and factors:

```
def parse_unary(tokens):
    if tokens[0]["tag"] == "-":
        operand, tokens = parse_unary(tokens[1:])
        return {"tag": "unary-", "operand": operand}, tokens
    return parse_factor(tokens)
```

The recursive call makes repeated negation work. The source `--3` becomes a unary-minus node whose operand is another unary-minus node. A parenthesized expression can also be negated:

```
-(3+4)
```

Unary minus is separate from binary subtraction. The tokenizer recognizes the same `-` token in both cases. The parser decides whether that token begins a unary expression or sits between two terms.

Parsing Terms and Expressions

The term parser collects multiplication and division:

```
def parse_term(tokens):
    left, tokens = parse_unary(tokens)
    while tokens[0]["tag"] in ["*", "/"]:
        op = tokens[0]["tag"]
        right, tokens = parse_unary(tokens[1:])
        left = {"tag": op, "left": left, "right": right}
    return left, tokens
```

First it gets a unary expression. As long as the next token is `*` or `/`, it consumes that operator and another unary expression, then creates a new operator node. The previous tree becomes the new node's left child. This lets multiplication and division use negated operands, as in `3*-2`.

This produces **left associativity**. In `8/4/2`, the first node represents `8/4`. The next node represents `(8/4)/2`. Its result is `1.0`, whereas `8/(4/2)` produces `4.0`.

The expression parser has the same structure at the next precedence level:

```
def parse_expression(tokens):
    left, tokens = parse_term(tokens)
    while tokens[0]["tag"] in ["+", "-"]:
        op = tokens[0]["tag"]
        right, tokens = parse_term(tokens[1:])
        left = {"tag": op, "left": left, "right": right}
    return left, tokens
```

For `3+4*5`, the first term is the number `3`. After reading `+`, the expression parser asks for another whole term. That term parser consumes `4*5` and returns its multiplication node. Only then does the expression parser construct the addition node.

Precedence and associativity are separate choices. Precedence decides how different operator levels group together. Associativity decides how a chain at one level groups. The function calls establish the levels; the loops build left-associated chains.

Requiring the Whole Input

The public `parse` function checks that parsing consumed the entire expression:

```
def parse(tokens):
    ast, tokens = parse_expression(tokens)
    if tokens[0]["tag"] is not None:
        raise SyntaxError(f"Unexpected token: {tokens[0]}")
    return ast
```

Without this check, `3 4` could be accepted as the expression `3`, with the second number silently ignored. A helper successfully parsing a prefix is not the same as the full input being valid.

Parentheses make that distinction necessary. While parsing the contents of `(3+4)`, the expression helper needs to stop before `)`, leaving it for the factor function. At the outermost level, `parse` instead requires the end marker.

Evaluating the Tree

The evaluator follows the AST recursively:

```
def evaluate(ast):
    if ast["tag"] == "number":
        return ast["value"]
    elif ast["tag"] == "+":
        return evaluate(ast["left"]) + evaluate(ast["right"])
    elif ast["tag"] == "-":
        return evaluate(ast["left"]) - evaluate(ast["right"])
    elif ast["tag"] == "unary-":
        return -evaluate(ast["operand"])
    elif ast["tag"] == "*":
        return evaluate(ast["left"]) * evaluate(ast["right"])
    elif ast["tag"] == "/":
        return evaluate(ast["left"]) / evaluate(ast["right"])
    else:
        raise ValueError(f"Unknown AST node: {ast}")
```

A number node is the base case. A binary operator node evaluates its children and applies the corresponding Python operation to their values. A unary-minus node evaluates its operand and negates that value. For our example, the left child supplies `3`; the right subtree supplies `20`; the addition returns `23`.

The evaluator does not need to know that `*` has higher precedence than `+`. The parser has already encoded the grouping in the tree. Nor does the evaluator need to find parentheses in the source.

This implementation uses Python's numeric behavior. `/` performs true division, so `6/2` produces `3.0`. Division by zero raises an exception. These are semantic choices inherited from the implementation, even though the input language has its own syntax.

Testing the Implementation

We do not yet have enough of Vertex to write a test suite in Vertex. There is no output statement or assertion facility. For now, we will be content with the unit tests in the Python code.

Run the existing Python unit tests from this chapter's code directory:

```
python3 tokenizer.py
python3 parser.py
python3 evaluator.py
```

Each file calls its test functions when run directly. Python assertions compare actual and expected results. An assertion failure interrupts execution; successful runs reach `done.`. These assertions belong to the implementation language, not to the expression language we are building.

The tokenizer tests cover integer and floating-point forms, operators, whitespace handling, and an unexpected character. Comment tests check line endings, comments at the end of input, and the positions of following tokens. They also check that ordinary division still works. Parser tests check node dictionaries and the unused tokens returned by helpers. Evaluator tests use both manually constructed trees and tokenized, parsed expressions.

Checking the tree matters even when a result looks right. Addition can conceal grouping errors: `(1+2)+3` and `1+(2+3)` both yield `6`. Subtraction and division make associativity mistakes easier to detect.

Additional cases worth checking are:

Input	Expected behavior	Stage to inspect
`3+4*5`	`23`	Precedence in parsing
`(3+4)*5`	`35`	Parenthesized factor
`8/4/2`	`1.0`	Left associativity
`0-5`	`-5`	Binary subtraction
`-5`	`-5`	Unary minus
`3*-2`	`-6`	Unary operand in multiplication
`.5+5.`	`5.5`	Floating-point literals
`3 4`	Syntax error	Complete-input check
`3+*4`	Syntax error	Missing operand
`(3+4`	Syntax error	Missing closing parenthesis
`3@4`	Unexpected-character error	Tokenization
`3/0`	Division-by-zero error	Evaluation

An error is not always evidence that the interpreter is broken. Rejecting an invalid program is part of its job. A useful test checks that rejection happens for the expected reason.

At this stage, the runner does not catch exceptions and reformat them. If you run one of these error cases through `./vertex`, you will see a Python traceback ending in the relevant error. Later versions can wrap those exceptions and print shorter language-level error messages.

Considerations

1. Which tests distinguish left-associative division from right-associative division? Why would addition make a weaker test?
2. What would change if the tokenizer treated a leading minus as part of every numeric literal? Consider both subtraction and negation.
3. Could this interpreter discard source positions without changing computed values? What would become harder?
4. Where would you add an exponentiation operator, and how would its precedence and associativity affect the parser?

Practice

Answers to the practice questions follow this section.

Work in a copy of the topic directory.

1. Run `example.v`, then change a copy to `(3+4)*5`. Trace why the result changes when the expression tree changes.
2. Print the tokens for `12 + 5`. Identify the positions, numeric values, and final marker.
3. Use Python's `pprint` to inspect the tree for `3+4*5`. Draw the tree for `(3+4)*5` and compare their roots.
4. Call `parse_factor` on the tokens for `3+4`. Inspect both return values. Repeat with `parse_term` on `3*4+5`.
5. Add Python unit tests for `8-3-2` and `8/4/2`. Check the tree structure as well as the result.
6. Add Python tests that expect errors for `3 4`, `(3+4`, and `3+*4`. Keep a test for division by zero separate from these syntax cases.
7. Construct a multiplication node manually and pass it directly to `evaluate`. Explain what this tests without involving the tokenizer or parser.
8. Add tests for `.5`, `5.`, and `12.5`. Check both the token values and the final evaluation results.
9. Add tests for `-5`, `3*-2`, and `-(3+4)`. Check that unary minus creates a `unary-` node rather than becoming part of the number token.

Answers to the Practice Questions

1. The original expression evaluates to `4.5`. `(3+4)*5` evaluates to `35`. The original root is `+`, with multiplication in its right subtree. The changed root is `*`, with addition in its left subtree.

2. The tokens are number `12` at line 1, column 1; `+` at column 4; number `5` at column 6; and the final marker, whose tag is `None`, at column 7. Whitespace does not produce a token. Number-token values are numbers, not the original digit strings.

3. `3+4*5` has `+` at the root and `*` below its right edge. `(3+4)*5` has `*` at the root and `+` below its left edge. Parentheses affect which subtree becomes an operand; they do not become nodes themselves.

4. `parse_factor(tokenize("3+4"))` returns a number node for `3` and the remaining `+`, `4`, and final-marker tokens. `parse_term(tokenize("3*4+5"))` returns a multiplication node for `3*4` and leaves `+`, `5`, and the final marker. Each function consumes only the grammar rule it handles.

5. Subtraction and division associate to the left. A test can check both the result and the left subtree:

```
from tokenizer import tokenize
from parser import parse
from evaluator import evaluate

for source, operator, expected in [("8-3-2", "-", 3), ("8/4/2", "/", 1)]:
    tree = parse(tokenize(source))
    assert tree["tag"] == operator
    assert tree["left"]["tag"] == operator
    assert evaluate(tree) == expected
```

6. Parsing each of `3 4`, `(3+4`, and `3+*4` raises `SyntaxError`. Test those calls with `assertRaises(SyntaxError)`. Separately, `1/0` parses normally and raises `ZeroDivisionError` only when evaluated. A syntax test should not count that runtime error as a successful parser rejection.

7. Evaluate `{"tag": "*", "left": {"tag": "number", "value": 3}, "right": {"tag": "number", "value": 4}}`. The result is `12`. This isolates the evaluator's multiplication rule from scanning and parsing.

8. `.5`, `5.`, and `12.5` each produce one number token, followed by the final marker. Their token values and evaluation results are `0.5`, `5.0`, and `12.5` respectively.

9. The results are `-5`, `-6`, and `-7`. The first and third trees have a `unary-` root. In `3*-2`, the multiplication node's right child is `unary-`. The minus sign is a separate token in all three cases.

References

1. Wikipedia contributors. "Regular expression." Wikipedia. https://en.wikipedia.org/wiki/Regular_expression
2. Wikipedia contributors. "Extended Backus-Naur form." Wikipedia. https://en.wikipedia.org/wiki/Extended_Backus%E2%80%93Naur_form
3. Wikipedia contributors. "Abstract syntax tree." Wikipedia. https://en.wikipedia.org/wiki/Abstract_syntax_tree
4. Wikipedia contributors. "Recursive descent parser." Wikipedia. https://en.wikipedia.org/wiki/Recursive_descent_parser